Projeto Final

Valor: 30 (+3) pontos
Data de entrega: 18/06/2026

1. Introdução

O grupo deve escolher um problema de seu interesse e realizar todo o processo de desenvolvimento (análise, projeto e implementação) de uma solução. Espera-se ao final um sistema de porte médio (>2000 linhas de código) que utiliza os conceitos e técnicas vistos durante o curso (modelagem, POO, testes unitários, etc). O programa deve ser feito baseado na linguagem **C++11**.

O desenvolvimento e a entrega deverão ser feitos utilizando o sistema de controle de versão GitHub[1]. Sugere-se que commits/pushs sejam feitos de maneira frequente, sempre que houver alterações.

O calendário de atividades do trabalho é mostrado abaixo:

Atividade	**Data**
Formação do Grupo (planilha Moodle)	Até 12/03/26
Definição do tema (planilha Moodle)	Até 17/03/26
Apresentação / Entrevista	16/06/26 e 18/06/26
Entrega final (Github)	18/06/26

2. Temas

Nesta seção são apresentados os temas disponíveis para o projeto final da disciplina. Para cada proposta, são descritas funcionalidades em **alto nível**, com o objetivo de orientar a modelagem inicial do sistema. Essas funcionalidades não devem ser encaradas como uma lista fechada de requisitos: os alunos **podem (e devem) propor extensões, melhorias e novas funcionalidades**, desde que coerentes com o escopo do projeto.

O sistema deve permitir a aplicação dos principais conceitos da disciplina, especialmente **Programação Orientada a Objetos** (encapsulamento, herança e polimorfismo), além de boas práticas, modularidade, tratamento de exceções e testes unitários, apresentando modelagem consistente e responsabilidades bem definidas. O objetivo não é apenas criar um programa funcional, mas um sistema **confiável, reutilizável e de fácil manutenção e extensão**, sendo valorizadas a aplicação adequada dos conceitos, a criatividade e a implementação de funcionalidades adicionais.

Por fim, recomenda-se a utilização de **arquivos de texto** para persistência de dados (por exemplo, armazenamento de cadastros, histórico de operações, configurações, etc.) ou para simulação de entradas do sistema. O uso de persistência simples permite exercitar manipulação de arquivos, organização de dados e tratamento de erros, sem introduzir complexidade adicional como bancos de dados externos.

[1] https://github.com/

- E-commerce: O desafio é desenvolver uma plataforma de comércio eletrônico capaz de gerenciar usuários, produtos e pedidos de forma organizada e segura, simulando o funcionamento básico de uma loja online.
    - Funcionalidades básicas: gestão de perfil de usuários, cadastro/listagem de produtos, carrinho de compras, efetivação e histórico de pedidos, relatórios/histórico, etc.

- Mini rede social: O desafio é construir uma mini rede social que permita modelar interações entre usuários e organizar conteúdos publicados em uma timeline.
    - Funcionalidades básicas: gestão de perfil de usuários, diferentes tipos de posts (e.g., texto, vídeo), listagem de posts em timeline, likes/comentários em posts, adicionar/listar amigos, etc.

- Estação Espacial: O desafio é desenvolver um sistema capaz de simular a gestão integrada de uma estação espacial, controlando módulos, tripulação e recursos críticos.
    - Funcionalidades básicas: diferentes tipos de módulos (e.g., habitação, laboratório), cadastro de astronautas e alocação e módulos, cadastro e gestão de recursos (e.g., oxigênio, energia, água), simulação e detecção de falhas em módulos, geração de relatórios de status da estação, etc.

- Sistema de Presença / “Ponto Eletrônico” de Alunos: O desafio é construir um sistema para o registro da presença dos alunos em diferentes matérias, departamentos, etc. de forma automatizada, utilizando o "identificador da carteirinha" (número hexadecimal de 48-bits), simulando o funcionamento de um sistema real de controle de acesso.
    - Funcionalidades básicas: Cadastro de diferentes tipos de alunos (e.g, Graduacão, Mestrado, Isolada, etc.), em diferentes tipos de curso (e.g, graduação, pós-graduacão), ter uma interface para validação da presença/carteirinhas dos alunos para cada dia de aula (modo aluno), uma interface para registro e edição dos alunos, cursos, etc (modo professor). Geração de relatórios (faltas por dia, por turma, por aluno, etc.).

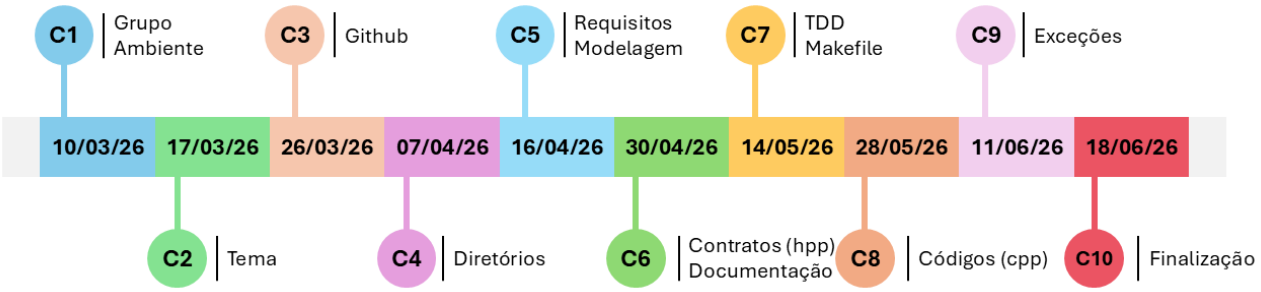
- Estacionamento Inteligente: O desafio é construir um sistema que simule o funcionamento de um estacionamento inteligente, controlando vagas, veículos e regras de cobrança.
    - Funcionalidades básicas: cadastro de diferentes tipos de vagas (e.g., Comum, Especial) e veículos (e.g., Carro, Moto, Caminhão), simular entrada/saída de veículos, calcular diferentes tipos de tarifas (considerar veículo, vaga, tempo), controle e alocação de vagas, geração de relatórios, etc.

- Jogo RPG (Turn-Based Game Engine Simplificado): O desafio é implementar um motor simplificado de RPG baseado em turnos, modelando personagens, regras de combate e gerenciamento de recursos.
    - Funcionalidades básicas: diferentes tipos de personagens, combate por turnos, regras de dano específicas por classe, inventário, gestão e uso de itens, etc.

- Engine para Jogos de Tabuleiro: O desafio é desenvolver uma engine genérica para jogos de tabuleiro, capaz de fornecer a infraestrutura básica para criação de diferentes jogos (por exemplo, damas, batalha naval, etc.), separando regras específicas da lógica geral do sistema.
    - Funcionalidades básicas: representação do tabuleiro, representação de peças e jogadores, sistema de turnos, estrutura para definição de regras específicas de cada jogo, mostrar ao menos dois jogos distintos feitos utilizando a mesma engine.

3. Checkpoints

O projeto será desenvolvido de forma **incremental**, possibilitando a evolução contínua da solução ao longo do semestre. O acompanhamento do trabalho ocorrerá por meio de **10 checkpoints**, distribuídos ao longo da disciplina, cada um associado a entregáveis específicos que permitem monitorar o progresso e realizar ajustes quando necessário. Os detalhes de cada checkpoint encontram-se descritos a seguir e no cronograma oficial da disciplina. O não cumprimento de um checkpoint acarretará **penalização de 5% na nota final**, cumulativa a cada etapa não atendida.



- **C1 – Formação do Grupo e Configuração do ambiente de desenvolvimento**
  - Preparação do ambiente de trabalho, incluindo instalação do compilador, bibliotecas, IDE e demais ferramentas necessárias para o projeto.
- **C2 – Definição do tema**
  - Escolha e delimitação inicial do escopo do projeto, com breve descrição do problema a ser resolvido e dos objetivos a serem alcançados.
- **C3 – Acesso ao GitHub e Commit de teste**
  - Configuração do repositório no GitHub e realização de um commit de teste para validação do fluxo de versionamento.
- **C4 – Estrutura de diretórios**
  - Organização inicial dos diretórios do projeto (ex.: src/, include/, build/, tests/) seguindo boas práticas de modularização.
- **C5 – Requisitos e modelagem**
  - Levantamento dos requisitos funcionais e não funcionais (User Stories) e elaboração da modelagem inicial (Cartões CRC) que descrevam o sistema.
- **C6 – Definição dos contratos (.hpp) e documentação inicial (README+Doxygen)**
  - Criação dos arquivos de cabeçalho com a especificação das interfaces e início da documentação automatizada usando Doxygen e descrição via README.
- **C7 – Testes de unidade (TDD) e Makefile**
  - Implementação inicial dos testes unitários seguindo a abordagem Test-Driven Development e criação do Makefile para automação de compilação.
- **C8 – Implementação inicial dos comportamentos (.cpp)**
  - Codificação inicial das funcionalidades principais do sistema com base nas interfaces definidas, garantindo conformidade com os requisitos.
- **C9 – Programação defensiva e Tratamento de exceções**
  - Inclusão de mecanismos para aumentar a robustez do código, prevenindo erros e tratando adequadamente condições excepcionais.
- **C10 – Finalização e apresentação**
  - Refinamento final do projeto, revisão de código, entrega da versão definitiva e apresentação dos resultados para a turma.

4. Comentários Gerais

- Comece a fazer o trabalho logo, enquanto o prazo está tão longe quanto jamais poderá estar.
- O trabalho deverá ser feito em grupos com 4 ou 5 alunos.
- Trabalhos copiados serão penalizados conforme anunciado.
- A entrega final será considerada o último commit na branch principal do projeto.
- Deve ser fornecido com o código um arquivo Makefile com as opções ‘make’ e ‘make run’.

5. Critérios de avaliação

5.1. Entrega Parcial (Requisitos e Modelagem)

- User Stories (3 pts).
- Cartões CRC (3 pts).

5.2. Entrega Final (Implementação)

- Documentação (4 pts).
  - Detalhamento do projeto (README).
  - Doxygen.
- Funcionamento correto (4 pts).
  - Compila e executa, não apresenta crash, etc.
- Uso correto das boas práticas e dos conceitos de OO (6 pts).
  - Abstração, Encapsulamento, Herança e Polimorfismo.
  - Modularização e componentes reusáveis.
- Programação defensiva / Tratamento de exceções (3 pts).
- Testes de Unidade / Cobertura (3 pts).
- Apresentação / Entrevista (4 pts).
- Participação individual ([0, 1]).
  - Baseado na <u>quantidade</u> de commits no GitHub.
  - **Grupo é responsável pela distribuição adequada de tarefas entre os membros!**
- Criatividade, extras (+3 pts).

Boolean de apresentação

$$\text{NotaFinal} = P \cdot (A \cdot \sum \text{Itens})$$

Float de participação